

Week 2: Product and Category Management System

Scenario

This project focuses on designing and managing product and category information using Oracle SQL. The system contains two related tables: CATEGORY and PRODUCT.

1. Design product and category table

Category Table:

```
SQL> INSERT INTO product VALUES('&product_id', '&product_name', '&cstegory_id', &pric, &stock_quantity);
Enter value for product_id: P1001
Enter value for product_name: LAPTOP
Enter value for cstegory_id: cat0001
Enter value for pric: 67500
Enter value for stock_quantity: 2
old 1: INSERT INTO product VALUES('&product_id', '&product_name', '&cstegory_id', &pric, &stock_quantity)
new 1: INSERT INTO product VALUES('P1001', 'LAPTOP', 'cat0001', 67500, 2)

1 row created.
```

Category table creation screenshot.

Definition: The CATEGORY table stores information about product categories. It contains a category ID, category name, and description. The category ID identifies each category uniquely.

Column	Data Type	Definition
cstegory_id	VARCHAR2(7)	Unique ID of a category.
category_name	VARCHAR2(20)	Name of the category.
description	VARCHAR2(50)	Details about the category.

Product Table:

```
SQL> create table product(product_id varchar2(7),product_name varchar2(20),category_id varchar2(7),pric number(9,2),stock_quantity number);
Table created.
```

Product table creation screenshot.

Definition: The PRODUCT table stores details about products. It contains a product ID, product name, category ID, price, and stock quantity. The category ID connects a product to its category.

Column	Data Type	Definition
product_id	VARCHAR2(7)	Unique ID of a product.
product_name	VARCHAR2(20)	Name of the product.
category_id	VARCHAR2(7)	ID of the related category.
pric	NUMBER(9,2)	Price of the product.
stock_quantity	NUMBER	Available quantity in stock.

2. Define Relationship and constraints

Relationship: CATEGORY and PRODUCT have a one-to-many relationship. One category can contain many products, while each product belongs to one category.

Defining relationship and creating constraint for product and category:

Primary key:

A primary key uniquely identifies each record in a table. It does not allow duplicate or NULL values.

```
SQL> ALTER TABLE category ADD CONSTRAINT PK_Category PRIMARY KEY (cstegory_id);
Table altered.

SQL> ALTER TABLE product ADD CONSTRAINT pk_product PRIMARY KEY (product_id);
Table altered.
```

Primary key constraint screenshot.

Foreign key:

A foreign key connects the PRODUCT table with the CATEGORY table. It ensures that every category ID in PRODUCT exists in CATEGORY.

```
SQL> ALTER TABLE product ADD CONSTRAINT fk_product_category FOREIGN KEY (category_id) REFERENCES category(cstegory_id);
Table altered.
```

Foreign key constraint screenshot.

Adding constraints

Not null:

The NOT NULL constraint ensures that a column cannot be left empty. It is applied to category_name and product_name.

```
SQL> ALTER TABLE category MODIFY category_name varchar(100) NOT NULL;
Table altered.

SQL> ALTER TABLE product MODIFY product_name varchar(150) NOT NULL;
Table altered.
```

NOT NULL constraint screenshot.

Check:

The CHECK constraint restricts the values entered into a column. The price must be greater than zero, and stock quantity must be zero or greater.

```
SQL> ALTER TABLE product ADD CONSTRAINT chk_price CHECK (pric > 0);
Table altered.

SQL> ALTER TABLE product ADD CONSTRAINT chk_stock CHECK (stock_quantity >= 0);
Table altered.
```

CHECK constraint screenshot.

Unique:

The UNIQUE constraint prevents duplicate values. It ensures that category names are not repeated.

```
SQL> ALTER TABLE category ADD CONSTRAINT uq_category_name UNIQUE (category_name);
Table altered.
```

UNIQUE constraint screenshot.

3. Insert values

Category Table:

```
SQL> INSERT INTO category VALUES ('&cstegory_id','category_name','&description');
Enter value for cstegory_id: cat0001
Enter value for description: electronics
old 1: INSERT INTO category VALUES ('&cstegory_id','category_name','&description')
new 1: INSERT INTO category VALUES ('cat0001','category_name','electronics')

1 row created.
```

Category table insertion screenshot 1.

```
SQL> INSERT INTO category VALUES ('&cstegory_id','&category_name','&description');
Enter value for cstegory_id: cat0002
Enter value for category_name: clothing
Enter value for description: men,women and kids apparels
old 1: INSERT INTO category VALUES ('&cstegory_id','&category_name','&description')
new 1: INSERT INTO category VALUES ('cat0002','clothing','men,women and kids apparels')

1 row created.
```

Category table insertion screenshot 2.

Product Table:

```
SQL> CREATE TABLE category(cstegory_id varchar2(7),category_name varchar2(20),description varchar2(50));
Table created.
```

Product table insertion screenshot 1.

```
SQL> select * from product;

PRODUCT
-----
PRODUCT_NAME
-----
CATEGOR      PRIC STOCK_QUANTITY
-----
P1001
LAPTOP
cat0001      67500          2

P2001
shoes
cat0002      1400          5

PRODUCT
-----
PRODUCT_NAME
-----
CATEGOR      PRIC STOCK_QUANTITY
-----

P3001
smart phone
cat0001      25000          5
```

Product table insertion screenshot 2.

Display the product Table

The SELECT command is used to display the records stored in a table. The command SELECT * FROM category; displays category records, and SELECT * FROM product; displays product records.

```
SQL> select * from category;
```

```
CSTEGOR
```

```
-----
```

```
CATEGORY_NAME
```

```
-----
```

```
DESCRIPTION
```

```
-----
```

```
cat0001
```

```
category_name
```

```
electronics
```

```
cat0002
```

```
clothing
```

```
men,women and kids apparels
```

```
CSTEGOR
```

```
-----
```

```
CATEGORY_NAME
```

```
-----
```

```
DESCRIPTION
```

```
-----
```

Displayed CATEGORY table.

```
SQL> select * from product;
```

```
PRODUCT
```

```
-----
```

```
PRODUCT_NAME
```

```
-----
```

```
----
```

```
CATEGOR
```

```
PRIC
```

```
STOCK_QUANTITY
```

```
-----
```

```
P1001
```

```
LAPTOP
```

```
cat0001
```

```
67500
```

```
2
```

```
P2001
```

```
shoes
```

```
cat0002
```

```
1400
```

```
5
```

```
PRODUCT
```

```
-----
```

```
PRODUCT_NAME
```

```
-----
```

Displayed PRODUCT table.

CRUD Operation

CRUD represents four basic database operations: Create, Read, Update, and Delete.

Operation	Description	SQL Example
Create	Adds a new record.	INSERT INTO product VALUES ('P3001','smart phone','cat0001',25000,5);
Read	Displays records.	SELECT * FROM product;
Update	Changes an existing record.	UPDATE product SET pric=26000 WHERE product_id='P3001';
Delete	Removes a record.	DELETE FROM product WHERE product_id='P3001';

Conclusion

The Product and Category Management System was designed using CATEGORY and PRODUCT tables. Their relationship was established using primary and foreign keys, and data integrity was maintained using NOT NULL, CHECK, and UNIQUE constraints. Values were inserted, tables were displayed, and CRUD operations were explained for managing the database.